

# Python基础应用

## Python函数与面向对象

1 任务背景

2 知识准备

3 任务要求

4 任务解析

- ☞ 函数、类和对象等相关知识是 Python 编程的核心概念，也是实现面向对象编程的基础。
- ☞ 通过学习函数，可以将复杂的任务分解为较小的、可重用的代码块，提高代码的可读性和可维护性。
- ☞ 类和对象则提供了一种组织和封装数据及相关操作的方式，使代码更加模块化和灵活。
- ☞ 这些知识的学习有助于构建大型、复杂的软件项目，并使代码更加高效、易于扩展和维护。



1 案例背景

2 知识准备

3 任务要求

4 任务解析

- ☁ 函数是一组封装好的代码块，其中包含了单一或者相关联的功能，可供开发者后续重复调用，减少开发的工作量，提升开发效率。
- ☁ Python本身已经提供大量丰富的函数，可在编写代码时直接调用。
- ☁ 开发者也可根据需要，按照函数编写规范自行定义函数，构建模块化的结构，扩展更多的功能。



## （1）函数的格式

函数的书写规范如下：

```
def 函数名(参数1,参数2...):
```

```
    <代码块语句>
```

示例1：用函数来判断要报销的费用是否超过预算。

思考：

- 1.如果要判断预算是否超标，需要传入哪些参数？
- 2.请描述函数实现的逻辑。



## （1）函数的格式

参考答案：

- ☞ 1. 传入两个参数：预算限额、费用。
- ☞ 2. 函数实现逻辑如下：
  - ☞ （1）如果费用 $\leq$ 预算限额，返回“没有超预算”
  - ☞ （2）如果费用 $>$ 预算限额，返回“超过预算”



示例代码说明：

- 函数以def关键词开头，其后为函数名 Judge\_fee，该行后面加冒号。
- 函数需要传递参数时，应放在圆括号中，参数之间用逗号分隔。
- 函数正式的内容是在冒号:后开始的，代码块在书写时要注意要缩进。
- 函数的第一行语句可以是字符串，用于说明该函数。当然，也可以用python的注释符#来进行说明。
- Return语句用于返回函数计算后的结果，便于程序直接引用该结果。

```
#1-5 2.1 函数的格式 示例1
#函数的定义
def Judge_fee(fee,budget):
    "判断是否超过预算"
    if fee<=budget:
        return "没有超预算"
    else:
        return "超过预算"
```

## （2）函数的参数和调用-必需参数

- 必需参数：默认情况下调用函数时，参数值和参数名称是按函数声明中定义的顺序匹配起来的，而且参数的数量也必须保持一致。
- 因此，在调用时应按照定义的顺序、数量进行赋值，才能得到预期的结果。

示例1：已知收入1000万，成本820万，要求通过定义函数来计算利润。

思考：

- 1.函数需要传入哪些参数？
- 2.函数需要返回什么结果？其计算公式是什么？



## (2) 函数的参数和调用-必需参数

参考答案:

- ☁ 1.传入两个参数: 收入、成本
- ☁ 2.需要返回计算的结果-利润, 其公式为: 收入-成本



示例代码说明：

- 收入为1000，成本为820，正确的利润应该是180。
- 在调用时，用Profit(1000,820)来赋值可以得到正确的结果。
- 如果参数值传递的顺序错了，如用Profit(820,1000)来赋值，返回的结果就是错的。

```
#1-5 2.1 函数的参数和调用 示例1
#函数的必需参数
def Profit(Revenue, Cost):
    #计算利润值并返回
    return Revenue - Cost

#假设收入=1000, 成本=820
#第一种调用顺序:
print("利润为:", Profit(1000, 820))

#第二种调用顺序:
print("利润为:", Profit(820, 1000))
```

利润为: 180

利润为: -180

- 关键字参数：在调用函数时，用关键字来标识参数，这样即使调用时参数的顺序与函数声明时不一致，Python 解释器依然能够用参数名匹配参数值。

示例2：现有两名员工的信息为：马元，年龄35岁，司龄5年，基本薪资8000元；张明，年龄25岁，司龄5年，基本薪资5000元。员工奖金的计算方法为：司龄大于10年的，奖金=基本薪资\*2；司龄在5-10年的，奖金=基本薪资\*1.5；司龄小于5年的，奖金=基本薪资。要求定义函数来计算员工的奖金，输出计算结果，并返回奖金。

思考：

- 1.函数需要传入哪些参数？
- 2.函数需要返回哪些参数？
- 3.请描述函数实现的逻辑。



参考答案：

- ☞ 1.函数需要传入四个参数：姓名、年龄、司龄、基本薪资。
- ☞ 2.函数需要返回的参数为：奖金。
- ☞ 3.函数实现逻辑：
  - ☞ （1）先传入四个参数：姓名、年龄、司龄、基本薪资
  - ☞ （2）用if-elif-else语句判断司龄在什么范围，并用对应的公式计算员工的奖金
  - ☞ （3）用print输出计算结果
  - ☞ （4）返回奖金的计算值。



示例代码说明:

- Employee(name,age,lenSrv,bsSlry)定义了函数的参数名称、数量和顺序。
- If语句的代码块通过判断司龄,来计算员工的奖金,此处不详细解释。
- Em1=Employee(age=35,lenSrv=5,name="马元",bsSlry=8000)是采用关键字参数的调用方法,python解释器匹配关键字来给参数赋值,因此参数顺序不同时调用也没问题。
- Em2=Employee("张明",25,bsSlry=5000,lenSrv=2)是采用必需参数和关键字参数混用的方式来调用。
- 在调用时不提供关键字,系统就默认按照定义的参数名、顺序来赋值;当提供关键字时,就按照关键字名称来匹配赋值。

#1-5 2.1 函数的参数和调用 示例2

#函数的关键字参数

```
def Employee(name, age, lenSrv, bsSlry):  
    #lenSrv为司龄, bsSlry为基本薪资  
    #根据司龄来计算奖金bonus  
    if lenSrv>=10:  
        bonus=bsSlry*2  
    elif lenSrv>=5:  
        bonus=bsSlry*1.5  
    else:  
        bonus=bsSlry  
    print("|姓名",name,"|年龄",age,"|司龄",lenSrv,\  
          "|基本薪资",bsSlry,"|奖金",bonus)  
    return bonus
```

#用关键字参数调用

```
Em1=Employee(age=35,lenSrv=5,name="马元",bsSlry=8000)
```

#也可以用必需参数和关键字参数混合调用

```
Em2=Employee("张明",25,bsSlry=5000,lenSrv=2)
```

|    |    |    |    |    |   |      |      |    |         |
|----|----|----|----|----|---|------|------|----|---------|
| 姓名 | 马元 | 年龄 | 35 | 司龄 | 5 | 基本薪资 | 8000 | 奖金 | 12000.0 |
| 姓名 | 张明 | 年龄 | 25 | 司龄 | 2 | 基本薪资 | 5000 | 奖金 | 5000    |

- ☁ 默认参数是指在调用函数时，如果参数没有传递值，就用参数中的默认值来替代。
- ☁ 在定义函数时，默认参数必须要有默认值。

示例3：税务会计在计算增值税时，需要用到收入、税率、抵扣项。有时，只需要计算税款，不需要考虑抵扣项，因此在定义函数时，可以将抵扣项设置为默认参数。假设收入为100万，税率为13%，请设计函数并编写代码来计算税款。

思考：

- 1.函数需要传入哪些参数？
- 2.函数需要返回哪些参数？
- 3.请描述函数实现的逻辑。



参考答案:

- ☞ 1.函数需要传入三个参数: 收入、税率、抵扣项。
- ☞ 2.函数需要返回的参数为: 税款。
- ☞ 3.函数实现逻辑:
  - ☞ (1) 先传入三个参数: 收入、税率、抵扣项。其中, 抵扣项应该有默认值0, 如果调用时没有传入抵扣项, 就默认为0。
  - ☞ (2) 计算税款, 公式为:  $\text{收入} \times \text{税率} - \text{抵扣项}$
  - ☞ (3) 返回税款



示例代码说明：

- calculate\_tax(income, tax\_rate, deduction=0)定义了函数名称及三个参数，其中deduction为默认参数，其默认值为0。
- tax = calculate\_tax(income, tax\_rate)是无deduction参数时的调用方式，因此默认其值为0。
- {tax:.2f}是将计算的税款tax按照2位小数的方式显示。
- tax = calculate\_tax(income, tax\_rate, deduction)是按照完整参数调用的，此时deduction的值就为5，而不是默认值0。

#1-5 2.1 函数的参数和调用 示例3

#函数的默认参数

#定义函数: calculate\_tax, 有三个参数

#其中deduction为默认参数, 默认值为0

```
def calculate_tax(income, tax_rate, deduction=0):  
    tax = income * tax_rate - deduction  
    return tax
```

# 示例用法

income = 100 # 收入

tax\_rate = 0.13 # 税率

# 无抵扣项时, 直接调用函数

tax = calculate\_tax(income, tax\_rate)

print(f"无抵扣项, 税款为: {tax:.2f}")

# 有抵扣项时, 指定参数值

deduction = 5 # 抵扣项金额

tax = calculate\_tax(income, tax\_rate, deduction)

print(f"有抵扣项, 税款为: {tax:.2f}, 抵扣项: {deduction}")

无抵扣项, 税款为: 13.00

有抵扣项, 税款为: 8.00, 抵扣项: 5

## (2) 函数的参数和调用-不定长参数

- ☁ 不定长参数也称可变参数，是指在定义函数时，参数个数不固定，可以根据实际需要传递相应数量的参数给函数，实现计算。
- ☁ 例如：函数`calculate_profit(*args)`中，`*args`为不定长参数。

示例4：假设公司2月份收入为1000万，成本为500万，费用为200万，所得税率为25%，请用不定长参数来设计函数并编写代码来计算当月的毛利润、营业利润、净利润。

当知道收入、成本时，计算的是毛利润，其计算公式=收入-成本；

当知道收入、成本、费用时，计算的是营业利润，其计算公式=收入-成本-费用

当知道收入、成本、费用、税率时，计算的是净利润，计算公式=（收入-成本-费用）\*（1-税率）

思考：

- 1.函数需要传入的完整参数有哪些？
- 2.函数需要返回的对应参数是什么？
- 3.请描述函数实现的逻辑。



参考答案:

- 1. 营业利润的计算传入的参数最多，因此完整的参数为：收入、成本、费用、税率。
- 2. 因为计算的结果不同，函数需要返回的对应参数为：毛利润、营业利润、净利润。这三个结算结果都可以用同一个参数返回。
- 3. 函数实现逻辑：
  - (1) 先传入参数，并判断传入参数的数量。
  - (2) 如果传入2个参数，计算毛利润：收入-成本。
  - (3) 如果传入3个参数，计算营业利润：收入-成本-费用。
  - (4) 如果传入4个参数，计算净利润：(收入-成本-费用) \* (1-税率)
  - (5) 如果传入的参数不是2,3,4这三种，判断为参数有错误
  - (6) 返回对应的利润值



## (2) 函数的参数和调用-不定长参数

示例代码说明:

- 函数`calculate_profit(*args)`中, `*args`为不定长参数。在函数被调用时, 带一个\*的不定长参数以元组 (tuple) 的方式导入, 存放所有未命名的参数。
- `financial_data=[]`定义了一个空列表变量, 后续用于存放传入的每一个参数。
- 在`for arg in args`循环中, 从不定长参数`args`中逐一读取每一个参数。
- `financial_data.append(float(arg))`是列表的`append`方法, 将读取的每一个参数转换为`float`类型, 并存入列表变量`financial_data[]`中。
- 在`if len(financial_data)==2:`中, `len()`函数用于判断传入的参数个数, 如果为2, 则调用毛利润的计算公式。
- `raise ValueError("参数列表长度必须为 2、3、4")`是在参数个数不符合要求时, 捕捉错误, 并抛出错误信息, 便于程序能正常运行。
- `raw_profit=calculate_profit(income, cost)`用于调用函数, 可以看到调用时有两个参数, 均传入函数中的不定长参数`*args`中。
- 另外两个调用类似, 但传入的参数数量不同, 不再重复描述。

```
#1-5 2.1 函数的参数和调用 示例4
#函数的不定长参数
#定义函数: calculate_profit, 为不定长参数
def calculate_profit(*args):
    # 定义一个空列表来存储参数
    financial_data = []

    print("args=", args, end=" ")
    # 遍历参数列表
    for arg in args:
        # 将每个参数转换为适当的数据类型并添加到列表中
        financial_data.append(float(arg))

    if len(financial_data)==2:
        # 如果有2个参数, 计算毛利润
        profit = financial_data[0] - financial_data[1]
    elif len(financial_data)==3:
        # 如果有3个参数, 计算营业利润
        profit = financial_data[0] - financial_data[1] - financial_data[2]
    elif len(financial_data)==4:
        # 如果有4个参数, 计算净利润
        profit = (financial_data[0] - financial_data[1] - financial_data[2]) \
            * (1 - financial_data[3])
    else:
        # 如果参数列表长度不为 2、3、4, 则抛出错误
        raise ValueError("参数列表长度必须为 2、3、4")

    # 返回计算结果
    return profit

# 假设参数列表为 (收入, 成本, 费用, 税率)
income = 1000
cost = 500
expenses = 200
tax_rate = 0.25
raw_profit=calculate_profit(income, cost)
print("|毛利润:", raw_profit)
Operating_profit=calculate_profit(income, cost, expenses)
print("|营业利润:", Operating_profit)
net_profit = calculate_profit(income, cost, expenses, tax_rate)
print("|净利润:", net_profit)

args= (1000, 500)|毛利润: 500.0
args= (1000, 500, 200)|营业利润: 300.0
args= (1000, 500, 200, 0.25)|净利润: 225.0
```

补充说明:

- 在函数定义中，也可以用带两个\*的参数\*\*args来定义不定长参数，此时传入的参数将以字典方式存储，示例如下：
- `def calculate_profit(**args):`
- 如果要遍历字典中的每一对元素，可采用下面的语句：
- `for key, value in args.items():`
- 如果要调用定义的函数，可参考示例如下：
- `raw_profit = calculate_profit(income=1000, cost=500)`
- 对应的\*\*args不定长参数为：`args= {'income': 1000, 'cost': 500}`



- 在函数中，用return <表达式> 语句退出函数，同时返回一个值。
- 如果在return后面没有写表达式，就返回None。

示例1：已知收入和成本，假设要计算利润，请用函数验证有返回值和无返回值的结果差异。

思考：

- 1.函数需要传入的参数有哪些？
- 2.函数如果有返回参数，应该是什么？如果没有返回参数，结果是什么？



参考答案：

- ☁ 1.函数需要传入的参数为：收入、成本。
- ☁ 2.函数有返回参数时，为利润；无返回参数时，为None。



示例代码说明：

- 函数cal\_profit1(invest,cost)中，用return profit结束，因此返回了计算的结果profit。
- 函数cal\_profit2(invest,cost)中，用return结束，实际上并没有返回计算的结果，故值为None。

#1-5 2.1 函数的返回值 示例1

#函数的返回值

```
def cal_profit1(invest,cost):  
    profit=invest-cost  
    return profit
```

```
def cal_profit2(invest,cost):  
    profit=invest-cost  
    return
```

```
invest=1000  
cost=800  
profit1=cal_profit1(invest,cost)  
print("return后有表达式:",profit1)  
profit2=cal_profit2(invest,cost)  
print("return后无表达式:",profit2)
```

return后有表达式: 200

return后无表达式: None

课堂练习：

1.参照上面的语句，在Jupyter中输入代码并运行，查看运行结果，并熟悉相关知识点。



思考题：

1. 下面代码的输出结果是什么？在jupyter中运行并验证。

```
#1-5 2.1函数 思考1
def cal_salary(bas_salary, att_score, perf_score):
    """
    计算薪酬的函数
    参数:
    bas_salary: 基本薪资, 单位为元
    att_score: 考勤得分, 满分10分
    perf_score: 绩效得分, 满分10分
    返回:
    salary: 薪酬, 单位为元
    """
    attendance_bonus = bas_salary * 0.2 * att_score / 10
    performance_bonus = bas_salary * 0.3 * perf_score / 10
    salary = bas_salary + attendance_bonus + performance_bonus
    return salary

# 调用函数, 计算薪酬
bas_salary = 6000 # 基本薪资, 单位为元
att_score = 9.5 # 考勤得分
perf_score = 8.2 # 绩效得分

salary = cal_salary(bas_salary, att_score, perf_score)
print(f"薪酬是: {salary}元")
```



思考题：

2. 请将本节不定长参数部分的示例代码改写，要求用带两个\*的参数\*\*args来定义不定长参数。



参考答案：

1.输出结果：薪酬是：8616.0元。

2.参考代码见右边。

```
#1-5 2.1 练习2
#函数的不定长参数
#定义函数：calculate_profit，为字典类型的不定长参数
def calculate_profit(**args):
    # 定义一个空列表来存储参数值
    financial_data = []
    # 遍历参数列表
    for key, value in args.items():
        # 将每个参数转换为适当的数据类型并添加到列表中
        financial_data.append(float(value))

    print("args=", args)
    if len(financial_data) == 2:
        # 如果有 2 个参数，计算毛利润
        profit = financial_data[0] - financial_data[1]
    elif len(financial_data) == 3:
        # 如果有 3 个参数，计算营业利润
        profit = financial_data[0] - financial_data[1] - financial_data[2]
    elif len(financial_data) == 4:
        # 如果有 4 个参数，计算净利润
        profit = (financial_data[0] - financial_data[1] - financial_data[2]) \
            * (1 - financial_data[3])
    else:
        # 如果参数列表长度不为 2、3、4，则抛出错误
        raise ValueError("参数列表长度必须为 2、3、4")

    # 返回计算结果
    return profit

income = 1000
cost = 500
expenses = 200
tax_rate = 0.25
raw_profit = calculate_profit(income=income, cost=cost)
print("毛利润:", raw_profit)
Operating_profit = calculate_profit(income=income, cost=cost, expenses=expenses)
print("营业利润:", Operating_profit)
net_profit = calculate_profit(income=income, cost=cost, expenses=expenses, \
                             tax_rate=tax_rate)
print("净利润:", net_profit)
```

类是Python中面向对象编程的核心概念。它是创建对象的模板，其定义了一组属性和方法，可以用来定义对象的行为。

对象是类的实例，通过创建对象可访问类的属性和方法。

使用类和对象可以使代码更加简洁、清晰、易于复用、扩展和维护。



类



对象



类：

- ↪ 类是用来描述具有相同属性和方法的对象的集合
- ↪ 类定义了该集合中每个对象所共有的属性和方法
- ↪ 类使代码更加模块化和可复用

对象：

- ↪ 对象是根据类创建的实例
- ↪ 对象包含了类定义的属性和方法
- ↪ 对象的状态即为属性的值
- ↪ 对象可通过执行类定义的方法来操作属性值



使用类和对象的好处：

- 👉 封装和数据隐藏：通过定义类，可以将数据（属性）和对数据的操作（方法）捆绑在一起。这种封装使得代码更加模块化和易于维护。
- 👉 代码复用：使用类和对象有利于在不同的程序中共享和复用代码。
- 👉 继承和多态：类支持继承，通过基类可派生出特定的子类，并实现代码的层次结构和多态性，使代码更加灵活和可扩展。
- 👉 面向对象编程：通过使用类和对象，可以更好地模拟现实世界中的对象和它们之间的交互，从而使代码更具可读性和可维护性。



- 类的书写格式为： class 类名:
- “#定义类的基本属性”部分，用来定义类的属性和初始值。有时为了简化，可省略此部分。
- “#定义构造方法\_init\_()”部分，用def \_\_init\_\_(self,...)来初始化对象的属性值，要求第一个参数为self。
- ”#定义方法“部分，用来定义类的操作方法，跟函数作用相同，但要求第一个参数为self。

类的书写规范如下：

```
class 类名:
    #定义类的基本属性
    属性1=""
    ....

    #定义构造方法_init_()
    def __init__(self, 属性1,属性2,.....):
        self.属性1 = 属性1
        .....

    #定义方法
    def 方法1(self, 参数1, ....):
        <代码块>
```

示例1：用类来定义一个人员，其属性包含姓名、年龄、性别，其方法为显示该人员的属性值。将该类实例化为具体的对象，其属性值为姓名=“张振江”，年龄=30，性别=“男”，并调用对象的方法。

思考：

- 1.现实中的人可抽象出哪些属性？
- 2.针对人有哪些方法？可结合属性来思考。
- 3.人和类有什么共性？
- 4.现实中的人和对象有什么共性？



参考答案：

- 1.现实中的人可抽象出大量属性，如：姓名、年龄、性别、民族、学历、籍贯、住址等。
- 2.针对人有很多方法，如果结合属性来介绍，有：修改姓名、调整年龄、修正性别、修正民族、调整学历、调整籍贯、调整住址、输出相关属性值等。
- 3.人和类的共性体现为：两者都可看做抽象的概念。人是抽象出的一种生物体概念，有很多属性和行为；类是定义对象的属性和行为的一种方法，也是抽象的概念。
- 4.现实中的人和对象的共性体现为：两者都是具体的事物。现实中的人是具体化的，每个人都有唯一的身份，有很多属性和行为；对象是类的实例化，也有具体的属性和方法。



示例代码说明：

- name="", age=0等语句用于定义类People的基本属性，并定义初始值。
- 类People中定义了两个方法，“\_\_init\_\_”和“display\_info”，都用def开头。
- 类中的方法要求第一个参数都默认设置为“self”。
- def \_\_init\_\_()是类中的构造方法，用来初始化对象的属性。
- Pepl = People("张振江",30, "男")是将类实例化为对象Pepl。
- Pepl.display\_info()是用实例化后的对象Pepl来调用其方法display\_info()，故输出的是方法中的打印结果。
- print(Pepl.name, Pepl.age)是用于查看对象Pepl的两个属性值name、age输出结果。通过对比类中的初始值，在实例化时已经发生了改变。

#1-5 2.2类的对象和方法 示例1

```
class People:
    #定义类的基本属性
    name=""      #姓名
    age=0        #年龄
    gender=""    #性别

    #定义构造方法__init__()
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

    #定义方法display_info()
    def display_info(self):
        print(f"姓名: {self.name}, 年龄: {self.age}, 性别: {self.gender}")

# 创建一个人员实例
Pepl = People("张振江", 30, "男")

# 显示人员信息
Pepl.display_info()
print(Pepl.name, Pepl.age)
```

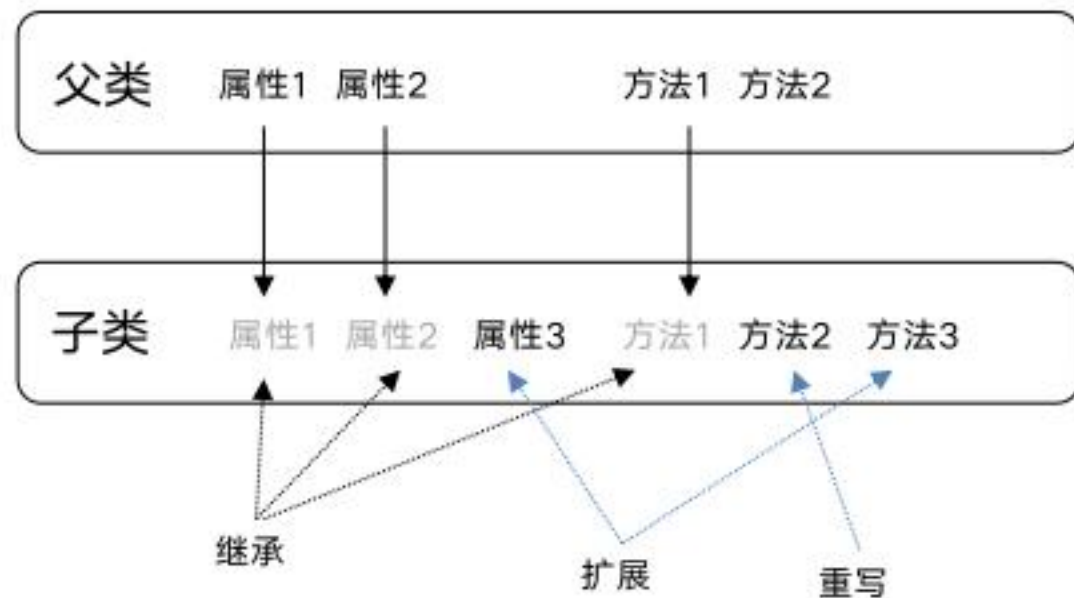
姓名: 张振江, 年龄: 30, 性别: 男  
张振江 30

- 在Python中允许创建一个新类（子类）来继承现有类（父类）的属性和方法
- 通过继承可重用、扩展或修改父类的代码，包括对属性、方法进行调用。

继承的书写方式为：

```
class 子类(父类):  
    <代码块>
```

在代码块中可继承父类的属性、方法，也可扩展子类的属性、方法，具体细节可参考后面的示例。



继承的主要价值：

- ☁ 重用代码：通过继承，可以重用父类的代码，而不需要从头开始编写。
- ☁ 扩展功能：子类可以继承父类的属性和方法，同时还可以添加自己特有的属性和方法，以实现对父类功能的扩展，满足特定的需求。
- ☁ 代码的可维护性：通过继承，可以将相关的类组织成类层次结构，这使得代码更加模块化和易于维护。
- ☁ 多态性：通过继承和方法重写，可以实现多态性，即不同的子类可以对相同的方法进行不同的实现，以灵活适应不同的需求。



示例1：对前面示例定义的类进行继承，并扩展定义为员工，其属性包含姓名、年龄、性别、部门、岗位、入职年份，其方法包含计算司龄，并显示该员工的姓名、岗位、司龄的属性值。将该类实例化为具体的对象，其属性值为姓名=“张振江”，年龄=30，性别=“男”，部门=“财务部”，岗位=“会计”，入职年份=“2016”，并调用对象的方法。

思考：

- 1.此案例中定义的员工和前一案例中定义的人有哪些不同？
- 2.如果要继承，可描述哪些属性和方法应继承？哪些应调整？
- 3.司龄的计算方法是什么？



参考答案：

1.此案例中定义的员工与前一案例中定义的人的不同点有：

- （1）人的概念范围更大，员工的概念范围相对较小
- （2）员工有特定的属性，如：部门、岗位、入职年份等

2.如果要继承：

- （1）员工可继承人的属性，如：姓名、年龄、姓名等，需要新增属性部门、岗位、入职年份
- （2）员工的方法中需要新增计算司龄的算法，但因为比较简单，可不用单独写函数。
- （3）员工的方法中也需要调整输出显示的内容。

3.司龄的计算方法：当前年份-入职年份。



## (3) 继承和方法重写

继承的示例代码说明：

- class People为父类，class Employee(People)为子类。
- 在子类class Employee的构造方法def \_\_init\_\_中，直接引用了父类的构造函数People.\_\_init\_\_(self,name,age,gender)来完成部分属性值的初始化。
- 在子类class Employee中，用self.Seniority = 2024-Year\_of\_Empty定义了工龄的计算方法，即用指定年份减去入职年份得到差值。
- 子类class Employee中新定义了一个方法def display\_Employee(self)，用于显示与员工相关的信息。
- 虽然在子类class Employee中没有定义方法display\_info()，但因为该子类继承了父类的方法，因此Empty对象用Empty.display\_info()可调用该方法。

#1-5 2.2 继承和方法重写 示例1

```
class People:
    # 定义类的基本属性
    name=""
    age=0
    gender=""

    # 定义构造方法 __init__()
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

    # 定义方法 display_info()
    def display_info(self):
        print(f"姓名: {self.name}, 年龄: {self.age}, 性别: {self.gender}")

class Employee(People):
    department="" # 部门
    post="" # 岗位
    Year_of_Empty=0 # 入职年份

    def __init__(self, name, age, gender, department, post, Year_of_Empty):
        # 调用父类的构造函数
        People.__init__(self, name, age, gender)
        self.department = department
        self.post = post
        self.Year_of_Empty = Year_of_Empty
        self.Seniority = 2024-Year_of_Empty # 工龄

    # 定义子类的方法
    def display_Employee(self):
        print(f"姓名: {self.name}, 岗位: {self.post}, 工龄: {self.Seniority}")

# 创建一个员工实例
Empty = Employee("张振江", 30, "男", "财务部", "会计", 2016)

# 显示员工信息
Empty.display_info()
Empty.display_Employee()
```

姓名: 张振江, 年龄: 30, 性别: 男  
姓名: 张振江, 岗位: 会计, 工龄: 8

示例2：将前面示例中的父类方法display\_info()进行重写，显示信息为姓名、部门、岗位。

思考：

1. 子类重写父类的方法，以下说法正确的是（ ）？

- A. 需要到父类中直接修改父类的方法
- B. 子类用跟父类同样的方法名，但实现内容不同
- C. 父类要用新的方法名
- D. 子类要用新的方法名



参考答案：

1.B

在重写父类方法时，子类只需要写一个与父类同名的方法，实现的内容跟父类不一样，就实现了对父类方法的重写。



## (3) 继承和方法重写

重写的示例代码说明：

- 在子类Employee中，定义了一个方法def display\_info(self)，该方法与父类的方法名称相同，因此重写了父类的方法。
- EmPLY.display\_info()是用对象EmPLY调用子类Employee的方法display\_info()，因此显示结果为姓名、部门、岗位。
- 如果要调用父类的方法，可以用示例中的super(Employee,EmPLY)函数来实现，函数中的Employee为子类，EmPLY为子类的对象。调用父类的display\_info()后，显示结果为姓名、年龄、性别。

#1-5 2.2 继承和方法重写 示例2

```
class People:
    # 定义类的基本属性
    name=""
    age=0
    gender=""

    # 定义构造方法 __init__()
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

    # 定义方法 display_info()
    def display_info(self):
        print(f"姓名: {self.name}, 年龄: {self.age}, 性别: {self.gender}")

class Employee(People):
    department="" # 部门
    post="" # 岗位
    Year_of_Emply=0 # 入职年份

    def __init__(self, name, age, gender, department, post, Year_of_Emply):
        # 调用父类的构造函数
        People.__init__(self, name, age, gender)
        self.department = department
        self.post = post
        self.Year_of_Emply = Year_of_Emply
        self.Seniority = 2024-Year_of_Emply # 司龄

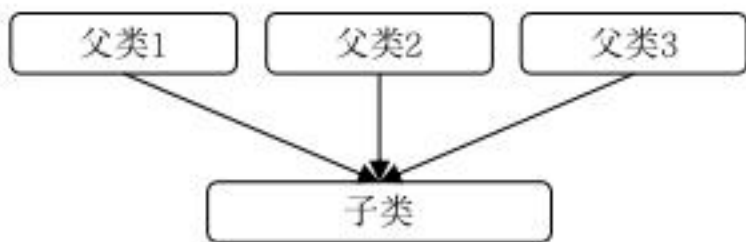
    # 定义子类的方法
    def display_Employee(self):
        print(f"姓名: {self.name}, 岗位: {self.post}, 司龄: {self.Seniority}")
    # 重写父类的方法
    def display_info(self):
        print(f"姓名: {self.name}, 部门: {self.department}, 岗位: {self.post}")

# 创建一个员工实例
EmPLY = Employee("张振江", 30, "男", "财务部", "会计", 2016)

# 显示员工信息
EmPLY.display_info() # 显示子类方法的结果
super(Employee, EmPLY).display_info() # 显示父类方法的结果

姓名: 张振江, 部门: 财务部, 岗位: 会计
姓名: 张振江, 年龄: 30, 性别: 男
```

- 多继承是指子类可以继承多个父类的属性和方法，这样可以复用较多的代码和方法
- 多继承的表示形式为：class 子类名称(父类1,父类2, 父类3...)
- 在定义子类时，父类的排列顺序将影响调用的结果。如果子类调用父类的方法，而多个父类都有同样的方法，则排在前面的父类的方法将首先被调用。



多继承的主要价值：

- ☁ 使用多继承，可以将类的编程结构化、层次化
- ☁ 共用的属性和方法在上层的父类中实现，个性化的属性和方法在下层实现
- ☁ 可以最大程度地重复利用代码，简化编程，并利于维护。



示例1：将前面示例的代码进一步改写，要求：

定义一个子类Major（专业），继承父类（Employee），并新增属性major（专业），重写父类的方法display\_Employee，只显示姓名、专业。

定义一个子类Certificate（证书），继承父类（People），并新增属性Certif（证书），新增方法display\_Employee，只显示姓名、证书。

定义一个子类Accountant1，用多继承的方式，父类顺序为（Major，Certificate）。再定义一个子类Accountant2，用多继承的方式，父类顺序为（Certificate，Major），以便对比查看两者的不同。

将EmPLY分别实例化为Accountant1和Accountant2的对象，并调用display\_Employee方法，对比查看两次调用的结果有何不同。EmPLY的相关参数为：姓名="张振江"，年龄=30，性别="男"，部门="财务部"，岗位="会计"，入职年份=2016，专业="财务管理"，证书="CPA"。

根据上面的要求编写代码，运行后查看和分析输出结果。

思考：

- 1.类Accountant1的输出的结果是什么？为什么？
- 2.类Accountant2的输出的结果是什么？为什么？
- 3.思考多继承有什么好处？



参考答案：

- 1.类Accountant1的输出结果是：姓名、专业，因为其父类为顺序为Major，Certificate，因此调用父类Major的输出方法，结果就为姓名、专业。
- 2.类Accountant2的输出结果是：姓名、证书，因为其父类为顺序为Certificate，Major，因此调用父类Certificate的输出方法，结果就为姓名、证书。
- 3.多继承可以继承多个父类的属性、方法，可以最大程度地复用代码，提升开发效率。同时可以使基于类的编程更加结构化和层次化，便于扩展和维护。



## (4) 多继承

多继承的示例代码说明（完整内容可查阅源代码）：

- 子类class Major定义了“专业”这个类，在定义时继承了父类“Employee”，因此在构造方法中直接调用了父类的构造函数。
- 子类class Major中定义了display\_Employee()方法。由于其父类Employee中也有同名的方法，因此实际上是重写了父类的方法。
- 子类class Certificate定义了“证书”这个类，在定义时继承了父类“People”，因此在构造方法中直接调用了父类的构造函数。
- 子类class Certificate中定义了display\_Employee()方法，该方法属于子类新增的方法。
- 子类class Accountant1继承了两个父类，为Major和Certificate。Accountant2也继承了两个父类，顺序不一样，为Certificate和Major。
- Emple实例化为类Accountant1的对象时，调用Emple.display\_Employee()是先调用父类Major中的方法，因此输出的结果是显示姓名、专业。
- Emple实例化为类Accountant2的对象时，调用Emple.display\_Employee()是先调用父类Certificate中的方法，因此输出的结果是显示姓名、证书。
- 类是可以进行多重继承的，如子类Employee继承了父类People，而子类Major又继承了父类Employee，实现了两重继承。

#1-5 2.2 多继承 示例1

```
class People: #定义基本类People-人员
    # 定义类的基本属性
    name=""
    age=0
    gender=""

    # 定义构造方法 __init__()
    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

    # 定义方法 display_info()
    def display_info(self):
        print(f"姓名: {self.name}, 年龄: {self.age}, 性别: {self.gender}")

class Employee(People): #定义类Employee-员工，并继承类People的属性和方法
    department="" # 部门
    post="" # 岗位
    Year_of_Emple=0 # 入职年份

    def __init__(self, name, age, gender, department, post, Year_of_Emple):
        # 调用父类的构造函数
        People.__init__(self, name, age, gender)
        self.department = department
        self.post = post
        self.Year_of_Emple = Year_of_Emple
        self.Seniority = 2024-Year_of_Emple # 司龄

    # 定义子类的方法
    def display_Employee(self):
        print(f"姓名: {self.name}, 岗位: {self.post}, 司龄: {self.Seniority}")
    # 重写父类的方法
    def display_info(self):
        print(f"姓名: {self.name}, 部门: {self.department}, 岗位: {self.post}")

class Major(Employee): #定义类Major-专业，并继承类Employee的属性和方法
    major="" # 专业

    def __init__(self, name, age, gender, department, post, Year_of_Emple, major):
        # 调用父类的构造函数
        Employee.__init__(self, name, age, gender, department, post, Year_of_Emple)
        self.major = major

    def display_Employee(self): # 重写父类方法display_Employee
        print(f"姓名: {self.name}, 专业: {self.major}")
```

## （5）类的私有属性和私有方法

- ☁ 在类中可以定义私有属性与私有方法，以便在类的内部使用，不允许外部使用或调用。
- ☁ 类的私有属性、私有方法都是以两个下划线开头，如\_\_secretAttr为私有属性，def \_\_secretFun()为私有方法。



## （5）类的私有属性和私有方法

示例1：人力资源部要对外披露公司员工的薪酬水平，但财务部要求不能透露每个员工的薪资数据，也不能透露员工的基本薪资以及薪资的计算方法。相关信息如下：

员工薪资=基本薪资+出勤奖金+绩效奖金=基本薪资\*（1+出勤得分/10\*出勤奖金系数0.2+绩效得分/10\*绩效奖金系数0.3）

基本薪资：财务部为8000，采购部为7000，行政部为5000，其他为6000。

判断薪资水平的两个标准基线：上限为9000，下限为6500。

员工有：张晓明，张振江，刘娴婷，马海松。

对应部门为：采购部，财务部，行政部，仓储部。

对应考勤得分为：8.5，9.5，7.2，8.8。

对应绩效得分为：8.9，9.3，8.3，7.9。

请根据上面的信息，用类的私有属性和私有方法来计算出每个员工的薪资，对外只反馈该员工是否处于某个薪资水平，并用完整代码来实现调用，显示结果。

思考：

- 1.要定义的类有哪些属性？私有属性是什么？
- 2.要定义的类有哪些方法？私有方法是什么？
- 3.思考类的私有属性和私有方法有什么好处？



参考答案：

1.要定义的类的属性有：姓名、岗位、考勤得分、绩效得分、薪资上限、薪资下限、基础薪资、薪资，其中基础薪资、薪资为私有属性。

2.要定义的类的方法有：

（1）计算薪资：根据题目提供的公式计算员工薪资

（2）判断薪资水平：根据员工薪资，判断其处于什么水平。

其中，计算薪资属于私有方法。

3.类的私有属性、私有方法可以把保密的数据、计算方法封装起来，不对外公开，起到保护作用。外部调用时，仅能访问可对外公开的属性、方法。



## (5) 类的私有属性和私有方法

示例代码说明（完整内容可查阅源代码）：

- 在类Employee中定义了两个私有属性\_\_bas\_salary和\_\_salary，外部不能访问这两个私有属性。
- 在类Employee中定义了私有方法\_\_cal\_salary()，外部不能访问这个方法。
- 类Employee中通过定义scope()方法来访问私有方法\_\_cal\_salary()，其计算结果可反馈给外部。
- 通过循环来实例化四个员工对象，并计算其薪资，获得薪资的水平范围。
- 用私有属性和私有方法既保护了公司内部的薪资信息，也对外提供了所需的信息。

#1-5 2.2 类的私有属性和私有方法 示例1

```
class Employee:

    name=""          #姓名
    post=""          #岗位
    att_score=0.0    #考勤得分，满分10分
    perf_score=0.0   #绩效得分，满分10分
    Upper=0          #判断薪资的上限
    Lower=0           #判断薪资的下限
    __bas_salary=0.0 #基础薪资，私有属性
    __salary=0.0     #薪资，私有属性

    def __init__(self, name, post, att_score, perf_score, Upper, Lower):
        self.name = name
        self.post = post
        self.att_score = att_score
        self.perf_score = perf_score
        self.Upper = Upper
        self.Lower = Lower
        self.__bas_salary = 0.0
        self.__salary = 0.0

    def __cal_salary(self): #私有方法
        if self.post == "财务部":
            self.__bas_salary = 8000
        elif self.post == "采购部":
            self.__bas_salary = 7000
        elif self.post == "行政部":
            self.__bas_salary = 5000
        else:
            self.__bas_salary = 6000

        attendance_bonus = self.__bas_salary * 0.2 * self.att_score / 10
        performance_bonus = self.__bas_salary * 0.3 * self.perf_score / 10
        self.__salary = self.__bas_salary + attendance_bonus + performance_bonus
        return self.__salary
```

课堂练习：

1.参照上面的语句，在Jupyter中输入代码并运行，查看运行结果，并熟悉类和对象的相关知识点。



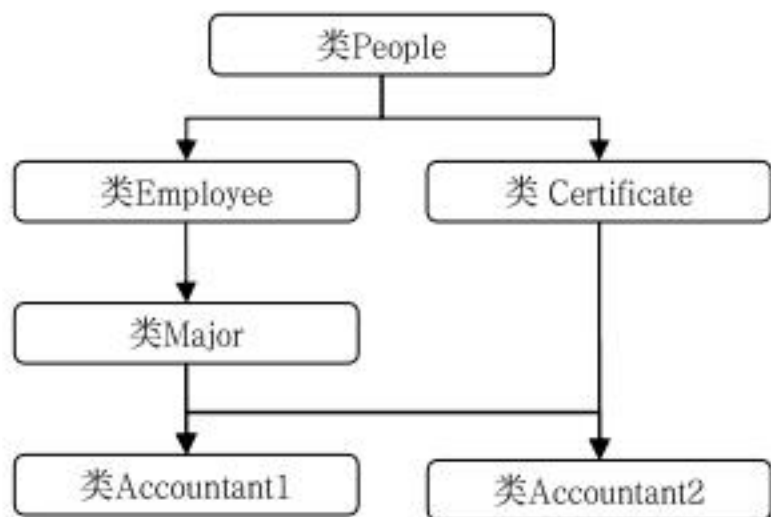
思考题：

- 1.请画出第（4）节多继承部分示例1中几个类的层级关系图。
- 2.尝试访问第（5）节示例1中的私有属性和私有方法，看系统反馈的结果是什么？



参考答案:

1. 多继承示例1中类的层级图如下:



参考答案：

2.尝试访问第（5）节示例1的私有属性时，系统反馈“'Employee' object has no attribute '\_\_bas\_salary'”，说明无法访问类Employee 的私有属性。

访问私有方法时，系统反馈“'Employee' object has no attribute '\_\_cal\_salary'”，说明无法访问类Employee 的私有方法。

1 案例背景

2 知识准备

3 任务要求

4 任务解析

已知华盛科技公司近三年的资产总额和负债总额，需要判断资产负债率是否有异常。公司所处行业的资产负债率的合理范围在40%到60%之间。要求如下：

- (1) 根据所学的函数知识，编写函数，并且通过调用该函数来判断公司三年的资产负债率是否正常。
- (2) 根据所学的类和对象的知识，用类和对象类来实现，判断公司三年的资产负债率是否正常。

相关的数据如下：

| 会计年度（年） | 2021        | 2022        | 2023        |
|---------|-------------|-------------|-------------|
| 资产总额（元） | 14824743567 | 15074434759 | 19146994680 |
| 负债总额（元） | 8668952922  | 13424278936 | 10641931399 |



1 案例背景

2 知识准备

3 任务要求

4 任务解析

思考：

- ☞ 1. 如果用函数来实现，函数需要传入哪些参数？函数要完成哪些计算任务？需要返回哪些值？
- ☞ 2. 如果用类和对象来实现，类应该定义哪些属性？定义哪些方法？
- ☞ 3. 用什么数据类型来定义三年的资产、负债数据？应该定义几个变量？
- ☞ 4. 用什么语法结构来复用语句并计算出三年的资产负债率？



参考答案：

- ☞ 1.如果用函数来实现，函数需要传入资产、负债两个数据。函数需要计算出资产负债率，并判断该值是否在正常范围内。函数需要返回判断的结果。
- ☞ 2.如果用类和对象来实现，类至少需要定义资产、负债两个属性。类应该要定义一个方法，该方法计算出资产负债率，并判断该指标是否在正常范围内，同时返回判断结果。
- ☞ 3.可以考虑用列表来定义三年的资产、负债数据，包含三个变量：年份列表、资产列表、负债列表。
- ☞ 4.可以用循环语句来复用语句，通过调用函数或者对象的方法，来计算出三年的资产负债率，并获得判断结果。



用函数实现的参  
考代码：

```
#1-5 3 任务要求-用函数实现
#判断资产负债率是否在正常范围的函数
def Asset_liability_ratio(asset,liability):
    Asset_liability_ratio=liability/asset
    #判断资产负债率
    if 0.4<Asset_liability_ratio<0.6:
        return "资产负债率正常"
    else:
        return "资产负债率异常"

#用列表表示数据
lst_year=[2021,2022,2023]      #年份数据
lst_asset=[14824743567,15074434759,19146994680]    #资产数据
lst_liability=[8668952922,13424278936,10641931399]  #负债数据

#用循环语句来计算资产负债率
i=0
while i<len(lst_year):
    IsNormal=Asset_liability_ratio(lst_asset[i],lst_liability[i])
    print(lst_year[i],IsNormal)
    i = i+1
```

2021 资产负债率正常  
2022 资产负债率异常  
2023 资产负债率正常

用类和对象实现  
的参考代码：

```
#1-5 3 任务要求-用类和对象来实现
class Asset_liability:
    Asset=0.0    #资产
    Liability=0.0    #负债

    def __init__(self, Asset, Liability):
        self.Asset=Asset
        self.Liability=Liability

    #判断资产负债率是否在正常范围
    def cal_Asset_liability_ratio(self):
        Asset_liability_ratio=self.Liability/self.Asset
        if 0.4<Asset_liability_ratio<0.6:
            return "资产负债率正常"
        else:
            return "资产负债率异常"

#用列表表示数据
lst_year=[2021, 2022, 2023]    #年份数据
lst_asset=[14824743567, 15074434759, 19146994680]    #资产数据
lst_liability=[8668952922, 13424278936, 10641931399]    #负债数据

#用循环语句来计算资产负债率
i=0
while i<len(lst_year):
    Ast_Liab=Asset_liability(lst_asset[i], lst_liability[i])
    IsNormal=Ast_Liab.cal_Asset_liability_ratio()
    print(lst_year[i], IsNormal)
    i = i+1
```

```
2021 资产负债率正常
2022 资产负债率异常
2023 资产负债率正常
```

作业1. 华盛科技记录了其客户的应收账款，要求利用所学的函数知识编写代码，计算每个客户的应收账款占比，并输出每个客户的名称、应收账款额、占比。相关数据如下：

| 客户   | 应收账款（万元） |
|------|----------|
| 宏达科技 | 500      |
| 隆江股份 | 1000     |
| 天工实业 | 750      |

作业2. 信息背景与上题一致，要求利用所学的类和对象知识，计算每个客户的应收账款占比，并输出每个客户的名称、应收账款额、占比。



感謝  
ขอบคุณ  
terima kasih  
ありがとう  
Thanks  
谢谢